

AI Task Management System

Complete Project Explanation & Interview Preparation Guide

Project Type: Python Full Stack Application | FastAPI + React + MySQL + JWT + RBAC + FAISS-based semantic retrieval

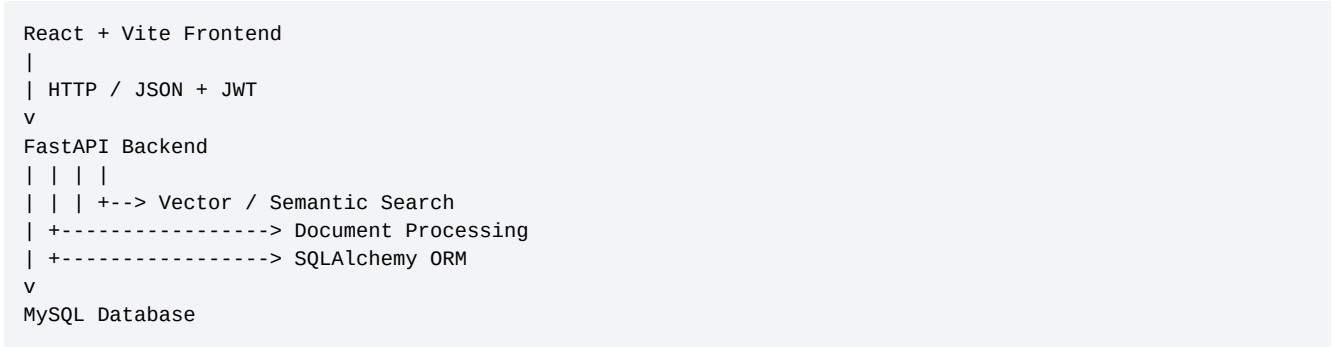
Purpose: A detailed interview-level explanation of the project architecture, modules, workflow, APIs, security, deployment lessons, testing, and common interview questions.

1. Project Overview

The AI Task Management System is a full-stack application designed to manage tasks, documents, user access, activity tracking, and AI-assisted task generation. The React frontend provides the user interface, while FastAPI exposes REST APIs for authentication, task operations, document processing, analytics, role-based access control, and audit logging.

- **Frontend:** React with Vite and Axios.
- **Backend:** FastAPI with router-based organization.
- **Database:** MySQL through SQLAlchemy ORM.
- **Authentication:** JWT bearer tokens.
- **Authorization:** RBAC using admin and user roles.
- **Documents:** PDF/TXT upload, extraction, chunking, embeddings, semantic search.
- **AI:** Retrieval-assisted task generation.
- **Audit:** Activity logging for important actions.

2. High-Level Architecture



The layered design keeps presentation, API logic, business services, and persistence concerns separate. This improves maintainability and makes modules easier to test.

3. Frontend Architecture

The frontend contains pages such as Login, Dashboard, Tasks, Documents, Analytics, and Audit Logs. React Router handles navigation. A centralized Axios client handles backend communication.

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://127.0.0.1:8000",
});
```

The environment variable allows the same frontend code to run locally and in production. The request interceptor reads `access_token` from `localStorage` and attaches it as a Bearer token.

```
const token = localStorage.getItem("access_token");
if (token) {
  config.headers.Authorization = `Bearer ${token}`;
}
```

The response interceptor handles 401 responses by clearing authentication data and redirecting to login, providing consistent session-expiry behavior.

4. Authentication Workflow

```
User enters credentials
↓
POST /auth/login
↓
Backend validates credentials
↓
JWT access token returned
↓
Frontend stores access_token
↓
Axios sends Bearer token
↓
Protected backend endpoint validates token
```

JWT authentication is stateless. The backend validates the token for each protected request instead of storing a traditional server-side session.

5. Backend Router Design

FastAPI routers group related endpoints and keep the application modular.

- Authentication: login and token generation.
- Users: user creation and admin-protected operations.
- Tasks: create, retrieve, filter, update status, delete.
- Documents: upload and semantic search.
- AI: generate structured task data from prompts.
- Analytics: system statistics.
- Audit logs: activity history.

6. Database and SQLAlchemy ORM

SQLAlchemy maps Python classes to database tables. The User model includes `id`, `name`, `email`, `password_hash`, `role`, `is_active`, and `created_at`. The role field supports authorization decisions.

The User model also has separate `created_tasks` and `assigned_tasks` relationships because a task creator and assignee can be different users. Separate foreign keys avoid ambiguity and accurately model the domain.

7. Dependency Injection and Current User

```
def get_current_user(
  credentials = Depends(security),
  db: Session = Depends(get_db)
):
```

```

token = credentials.credentials
payload = decode_access_token(token)
user_id = payload.get("sub")
...
return user

```

- HTTPBearer extracts the bearer token.
- The JWT is decoded.
- The user ID is read from the token subject.
- The user is loaded from MySQL.
- Existence and active status are checked.
- The authenticated user is injected into the endpoint.

8. Role-Based Access Control (RBAC)

Authentication answers 'Who are you?' Authorization answers 'What are you allowed to do?' The project separates these concerns. `get_current_user` validates identity; an additional dependency such as `get_current_admin` checks the role for administrative routes.

Interview answer: I implemented RBAC at the backend dependency level so hiding a frontend button is not treated as security. The backend makes the final authorization decision.

9. Task Management Module

Tasks support listing, filtering, creation, status updates, and deletion. The React page manages loading, error, success, filter, and operation-specific states.

- `tasks` stores fetched tasks.
- `loading` controls loading UI.
- `status` and `priority` control filters.
- `showCreateForm` controls the form.
- `creating`, `updatingId`, `deletingId` prevent duplicate requests.

Status transitions include `pending` → `in_progress` and `completed`. After mutations, the frontend refreshes data so the UI reflects backend state.

10. Document Upload and Processing

```

extension = Path(file.filename).suffix.lower()

if extension not in [".pdf", ".txt"]:
    raise HTTPException(
        status_code=400,
        detail="Only PDF and TXT files are supported"
    )

text = extract_text(str(file_path))
chunks = chunk_text(text)
create_embeddings(chunks)

```

Chunking divides long text into smaller units. Embeddings convert chunks into numerical vectors, allowing meaning-based retrieval instead of exact keyword matching.

11. FAISS / Semantic Search

The vector service exposes functions such as `create_embeddings` and `search_similar`. A query is compared against indexed document chunks and the most relevant content is returned.

Interview explanation: Traditional keyword search matches words. Semantic search tries to retrieve content with similar meaning. This is useful when the user and document use different wording for the same concept.

Production consideration: A local FAISS index is convenient for development, but durable storage or a managed vector database is preferable for scalable production deployment.

12. AI Task Generator

```
results = search_similar(prompt, 3)

context = ""

if results:
    context = results[0]["content"]

return {
    "title": prompt,
    "description": (
        f"Task generated from uploaded document context: {context}"
        if context
        else "Task generated from AI prompt"
    ),
    "priority": "medium",
    "due_date": None
}
```

The implementation first retrieves relevant document context and then creates a structured response. If no relevant context exists, it uses a fallback description.

Honest interview point: The current implementation is retrieval-assisted structured generation. It should not be described as a direct external LLM integration unless an actual LLM API is added.

13. Audit Logging

The `ActivityLog` model stores `user_id`, `action`, `details`, and `created_at`. Implemented examples include `DOCUMENT_UPLOAD` and `DOCUMENT_SEARCH`.

```
activity = ActivityLog(
    user_id=current_user.id,
    action="DOCUMENT_UPLOAD",
    details=f"Uploaded document: {file.filename}"
)

db.add(activity)
db.commit()
```

Audit logs provide accountability, debugging support, security traceability, and visibility into important user actions.

14. Error Handling

Expected backend errors use `HTTPException` with appropriate status codes. Examples include invalid filenames, unsupported file types, invalid tokens, missing users, and inactive accounts. The frontend catches `Axios` errors and displays backend details or user-friendly fallback messages.

15. Testing Strategy

- Swagger UI for interactive API testing.
- Postman for saved collections and negative test cases.
- Browser testing for complete workflows.
- Database verification for persisted tasks, users, and logs.

Recommended test sequence: Login → protected endpoint → RBAC admin endpoint → create task → update task → delete task → upload document → semantic search → AI task generation → audit log verification.

16. Deployment Architecture

```

User Browser
|
v
Vercel - React Frontend
|
| VITE_API_URL
v
FastAPI Backend
|
+--> MySQL
+--> Document Storage
+--> Vector Index

```

A major deployment lesson was that a local URL such as `mysql+pymysql://...@127.0.0.1:3306/...` cannot be used by a cloud backend. In production, `127.0.0.1` refers to the cloud server itself, not the developer's computer. A network-accessible database and environment-based configuration are required.

17. Environment Variables

- `DATABASE_URL` for production database connectivity.
- JWT secrets and security configuration.
- `VITE_API_URL` for the frontend backend URL.
- Future third-party service keys.

Secrets and passwords should never be committed to a public repository. If a credential is exposed, it should be changed or rotated.

18. Security Considerations

- Store password hashes, not plain passwords.
- Validate JWT tokens on protected endpoints.
- Reject inactive users.
- Enforce RBAC in backend dependencies.
- Restrict CORS to trusted production origins.
- Validate uploaded file type and preferably MIME type and size.
- Use environment variables for secrets.
- Avoid logging sensitive information.

19. Performance and Scalability

- Large document extraction and embeddings can be slow.
- Embedding work can move to background jobs.
- Pagination reduces unnecessary task-list payloads.
- Database indexes improve common queries.
- Persistent vector storage is preferable for production.
- Object storage is preferable to temporary local upload folders.

20. Key Design Decisions

FastAPI: Fast development, Pydantic validation, dependency injection, automatic OpenAPI docs.

React: Component-based UI and state-driven rendering.

SQLAlchemy: ORM abstraction and relationship management.

JWT: Stateless API authentication.

RBAC: Different permissions for different roles.

Vector search: Meaning-based document retrieval.

Audit logs: Traceability and accountability.

21. Interview Questions and Strong Answers

Q: Explain the project in 60 seconds.

A: I built an AI-powered task and knowledge management system using React, FastAPI, MySQL, and SQLAlchemy. Users authenticate with JWT, manage tasks, upload documents, and search document content semantically using a vector retrieval pipeline. I also implemented role-based access control and activity logging. The frontend communicates with FastAPI through Axios interceptors while the backend is organized into routers, services, models, schemas, and dependencies.

Q: How does authentication work?

A: The user logs in, the backend validates credentials and returns a JWT access token. The frontend stores it and Axios attaches it as a Bearer token. FastAPI extracts and decodes the token, then loads the authenticated user from the database.

Q: How does RBAC work?

A: Authentication is handled first with `get_current_user`. Admin-only endpoints add another dependency that verifies `current_user.role`. This keeps identity validation separate from authorization.

Q: How does document search work?

A: Text is extracted, chunked, converted into embeddings, and indexed. The query is compared semantically against indexed chunks and the most relevant results are returned.

Q: What deployment issue did you face?

A: The backend required `DATABASE_URL`, but the existing database URL pointed to `127.0.0.1`. I identified that localhost is not reachable from a cloud service and that production needs a cloud-accessible database plus environment configuration.

Q: What would you improve?

A: I would add background processing, persistent object storage, durable vector storage, automated tests, centralized audit logging, stronger file validation, and production environment separation.

22. Honest Limitations

- Current AI task generation is retrieval-assisted structured generation unless an external LLM API is explicitly integrated.
- Local uploads and local vector indexes need persistent production storage.
- A local MySQL instance cannot be reached from a cloud backend using localhost.
- Production requires database hosting, CORS, environment variables, and secret management.
- Frontend visibility controls are not a replacement for backend authorization.

23. Recommended Interview Demo Flow

- Explain Login and JWT.
- Show Dashboard navigation.
- Create and update a task.
- Demonstrate filters.
- Upload PDF/TXT.
- Search document content.
- Demonstrate AI Task Generator.
- Show analytics and audit logs.
- Explain admin RBAC.
- Finish with architecture, deployment lessons, and improvements.

24. Final Summary

This project demonstrates end-to-end Python full-stack development by combining frontend state management, REST APIs, JWT authentication, SQLAlchemy models, MySQL persistence, RBAC, document processing, semantic retrieval, AI-assisted task generation, and audit logging.

Strong closing statement: This project helped me understand how multiple layers of a full-stack application work together—from React UI state and API communication to FastAPI dependencies, authentication, database relationships, semantic retrieval, and deployment concerns. My focus was not only to implement features, but also to understand the engineering decisions and trade-offs behind them.